



Автосад

Интеллектуальное планирование
городского озеленения

Документация MVP для финальной сдачи

Назначение · Алгоритм · Архитектура
Docker · API · Проверка · Ограничения

29 сентября 2026

Подготовлено по README репозитория. Наличие реализации не заменяет проверку полного прогона на сдаваемом участке и в целевом CAD.

Интеллектуальное планирование городского озеленения.

Автосад помогает проектировщику перейти от CAD-подосновы к плану посадки деревьев и кустарников с учётом зданий, дорог и инженерных сетей. Сервис рассчитывает допустимые зоны, размещает растения, проверяет каждую посадку и возвращает новый DXF вместе с объяснимым отчётом. Исходный файл сохраняется отдельно и не перезаписывается.

Проект создан для кейса Департамента природопользования и охраны окружающей среды Москвы. Это MVP для локальной работы проектировщика, а не система автоматического согласования проектов. Главный результат - **DXF** → **расчёт** → **DXF + объяснения**.

Подготовка к финальной сдаче и соответствие ТЗ · Демофайл и пошаговый прогон · Документация в PDF · Настройки окружения · Docker Compose

Что умеет MVP

- Загружать DXF, анализировать слои, сущности, единицы, блоки и внешние ссылки.
- Предлагать классификацию слоёв и границу участка с подтверждением пользователя.
- Строить отдельные допустимые и запрещённые зоны для деревьев и кустарников.
- Генерировать посадки детерминированным алгоритмом или помощником на базе LLM.
- Проверять границу, нормативные отступы и межпосадочные интервалы независимым Validator.
- Показывать план на 2D-карте, добавлять, переносить и удалять посадки с серверной проверкой.
- Сохранять версии конфигурации и плана, повторять расчёт после изменения параметров.
- Выгружать DXF, JSON-план и отчёт в JSON/Markdown с проверками и нормативными ссылками.

Полный рабочий входной формат MVP - **DXF**. API также принимает DWF на хранение, но автоматический конвертер DWF → DXF не реализован; такой файл нельзя передать на анализ. UI принимает DXF. DWG, SHP и GeoJSON напрямую не импортируются.

Быстрый запуск в Docker

Нужны Git, Docker Engine и Docker Compose v2. Целевая среда по ТЗ - МосТех.ОС x86-64 / совместимый Linux. Dockerfile использует Linux-образы; отдельная проверка на МосТех.ОС пока не зафиксирована. Для рабочей станции ТЗ задаёт от 16 ГБ RAM, CPU от 3 ГГц и GPU от 4 ГБ для CAD; для тяжёлых чертежей рекомендуется 32 ГБ RAM. Само расчётное ядро не требует GPU.

В корне клонированного репозитория:

```
# Только при первом запуске: не перезаписывайте существующий .env.
cp .env.example .env

docker compose up --build -d
docker compose ps
docker compose logs --tail=100 migrate api worker
curl --fail http://127.0.0.1:8000/health
```

Compose запускает PostgreSQL, однократно применяет Alembic-миграции, затем запускает API со встроенным frontend и отдельный worker. Нормальное завершение migrate с кодом 0 ожидаемо; API и БД должны стать healthy, worker - оставаться запущенным. При первом запуске нужен интернет для получения образов и зависимостей.

Адрес по умолчанию	Назначение
localhost:8000	Веб-интерфейс
localhost:8000/docs	Swagger: вызов API из браузера
localhost:8000/openapi.json	Спецификация OpenAPI
localhost:8000/health	Проверка доступности API

Порт API задаётся `SERVER_PORT` в `.env`. По умолчанию порты доступны только на локальном компьютере. Метаданные хранятся в volume `postgres_data`, исходники и экспорты - в `file_storage`. Команда `docker compose down` останавливает сервис с сохранением этих данных; добавление `-v` удаляет volumes и данные.

Важная настройка для демо: в `.env.example` помощник выключен (`AGENT_ENABLED=false`), а кнопка генерации в UI вызывает именно помощника. Для полностью локального расчёта без LLM используйте маршрут генерации в Swagger, описанный ниже. Для работы кнопки настройте LLM по разделу «Помощник».

Прогон DXF до результата

1. Откройте интерфейс, создайте проект и загрузите подготовленный DXF.
2. Запустите анализ. Дождитесь завершения и изучите предупреждения о единицах, внешних ссылках и неподдерживаемых объектах.
3. Подтвердите единицы измерения и границу участка. Назначьте слоям смысловые типы, роль геометрии и необходимые атрибуты. Например, контур здания должен быть площадным объектом, а линия сети должна соответствовать указанному способу измерения расстояния. Не исключайте неизвестную коммуникацию ради успешного расчёта.
4. Выберите параметры: количества деревьев и кустарников, интервалы и доступные настройки ассортимента. Сохраните конфигурацию.
5. Запустите помощника в UI, если он настроен, **либо** в Swagger выполните `POST /api/projects/{project_id}/plans` без тела запроса. Этот маршрут запускает детерминированный алгоритм без LLM и без передачи данных внешнему провайдеру.
6. Сохраните `id` полученной задачи и опрашивайте `GET /api/jobs/{id}` до `succeeded` или `failed`. В успешном результате генерации будет `plan_id`. После API-генерации обновите страницу проекта, чтобы увидеть текущий план.
7. Выберите посадку на карте и откройте её проверки: фактическое расстояние, требуемое значение, результат, нормативный документ и пункт. При необходимости исправьте посадки и повторите проверку актуальной ревизии.
8. Сформируйте экспорт и скачайте весь комплект. Если доступен только черновой экспорт, он будет явно помечен и не означает нормативную готовность проекта.
9. Откройте DXF в целевом CAD, включите и выключите слои результата; проверьте сохранность подосновы, читаемость символов и соответствие посадок отчёту.

Подходящий DXF должен содержать выбранную границу, распознаваемую геометрию и однозначно классифицированные объекты. На финальное демо нужен заранее проверенный входной файл и сохранённый выходной комплект. Состояние примеров и план подготовки перечислены в чек-листе сдачи.

В репозитории есть синтетический DXF и готовая конфигурация. Подробная инструкция демо описывает слои и расчёт на части участка выше дороги (4 200 м²). Локальный прогон ядра с этой конфигурацией дал 12 деревьев, 20 кустарников и 96 успешных геометрических проверок; полный серверный экспорт и просмотр в CAD проверяются отдельно.

Как работает алгоритм



Подготовка. `ezdxf` разбирает сущности и вложенные INSERT-блоки. Линии, полилинии, дуги и окружности преобразуются в 2D-геометрию; кривые аппроксимируются с заданной погрешностью. Координаты переводятся в локальные метры. Это не WGS84 и не карта с географическими широтой и долготой. Некорректные контуры и неизвестные данные фиксируются как проблемы, а не исправляются молча.

Ограничения. Для каждого классифицированного объекта выбираются применимые правила по типу объекта, типу растения и атрибутам. `Shapely` строит буферы запретов и вычитает их из границы участка. Зоны деревьев и кустарников рассчитываются отдельно. Отсутствующее правило не трактуется как разрешение. Запись справочника об отсутствии числового требования не превращается в выдуманный нулевой отступ.

Алгоритмическая генерация. `hex-grid-greedy/1` перебирает несколько смещений гексагональной сетки. Сначала отбирает деревья, затем кустарники, соблюдая лимиты и межпосадочные интервалы. Из вариантов выбирает лучший по количеству деревьев, затем кустарников, с устойчивым порядком разрешения равенства. Это эвристика, не доказательство глобального оптимума; при недостатке места возможен недобор.

Гибридная генерация. Python готовит допустимые кандидаты и их характеристики; LLM выбирает композицию рядов, аллей и групп. Выбор проверяется в Python. Повторные обращения запрашивают недостающее количество; после исчерпания попыток Python может дополнить композицию, фиксируя такие добавления в предупреждениях. Модель не задаёт нормативные расстояния. Внешняя LLM не гарантирует побитовую воспроизводимость; для повторяемого расчёта предусмотрен алгоритмический режим.

Валидация. Независимый Validator проверяет каждую опубликованную посадку по исходной расчётной геометрии: принадлежность границе, расстояния до объектов, интервалы между растениями. Генерация, ручные правки и экспорт защищены проверками прикладного слоя, независимо от UI и инструкций модели.

Реализация: CAD, геометрия, генератор и Validator, помощник.

Нормы и объяснимость

Правила хранятся в `config/normative_rules.yaml`, ассортимент - в `config/plant_catalog.yaml`. В текущем справочнике 13 правил с источниками по постановлению Москвы № 743-ПП, п. 3.6.3, таблице 3.6.1; каталог содержит 248 записей. Версии, ссылки на публикации, даты проверки и ограничения применимости записаны в самих YAML-файлах.

Это описание текущих данных репозитория, а не заявление о полном покрытии всех норм ТЗ. СП 42.13330.2016, 623-ПП, специальные требования МГСН и весь набор охранных зон не реализованы в полном объёме. Их применимость к сдаваемому участку нужно проверить отдельно. Статус `verified` относится к реализованным проверкам и подтверждённым входам, а не к согласованию проекта уполномоченным органом.

Трассировка результата:

UUID посадки → проверяемый объект → фактическое / требуемое расстояние
→ правило → документ → пункт → статус и причина

Для каждой проверки сохраняются `planting_id`, тип и статус проверки, фактическое и требуемое значения, единица измерения, причина и, для нормативной проверки, идентификатор правила, документ, пункт и источник. Геометрическая принадлежность границе и проектные межпосадочные интервалы не выдаются за нормативные пункты. JSON-отчёт содержит также версии и SHA-256 исходника, конфигурации и справочников.

Состояние	Что означает для экспорта
План <code>verified</code> , <code>Validator passed</code>	Доступен обычный экспорт текущей ревизии
<code>needs_verification</code>	Возможен явно запрошенный демонстрационный черновик
Геометрические нарушения / невалидный план	Экспорт блокируется, включая черновой

После изменения конфигурации нужен новый расчёт. После правки посадок нельзя использовать проверку старой ревизии как подтверждение новой.

Экспорт и слои DXF

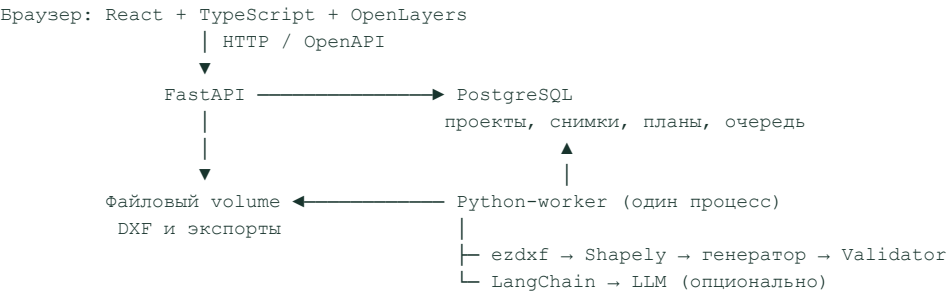
Комплект одной ревизии состоит из `result.dxf`, `plan.json`, `report.json` и `report.md`. Для черновика имена имеют префикс `draft-`, отчёты и DXF содержат предупреждение. Готовые файлы публикуются как неизменяемые артефакты.

Слой	Содержимое
<code>GREENPLAN_TREES</code> , <code>GREENPLAN_BUSHES</code>	Символы деревьев и кустарников
<code>GREENPLAN_TREE_LABELS</code> , <code>GREENPLAN_BUSH_LABELS</code>	Короткие марки Д-001, К-001
<code>GREENPLAN_LEGEND</code>	Экспликация марок и пород
<code>GREENPLAN_TREE_AVAILABLE</code> , <code>GREENPLAN_BUSH_AVAILABLE</code>	Допустимые зоны
<code>GREENPLAN_TREE_EXCLUSION</code> , <code>GREENPLAN_BUSH_EXCLUSION</code>	Зоны запретов
<code>GREENPLAN_DRAFT</code>	Предупреждение в черновом экспорте

Расчётные зоны по умолчанию выключены в DXF для читаемости плана. UUID, тип, порода и марка записываются в ATTRIB блока. Символ растения не изображает размер его кроны. Если имя слоя уже занято, создаётся уникальное имя; фактические имена включены в `plan.json`. Префикс `GREENPLAN_` сохранён как технический идентификатор после переименования сервиса в «Автосад».

Экспорт записывает новый файл и повторно читает его через `ezdxf`. Такая проверка не заменяет визуальный просмотр в целевом CAD.

Архитектура



HTTP-обработчики вызывают `services` и `repositories`; расчётное ядро не зависит от HTTP. Тяжёлые операции выполняет отдельный последовательный worker. PostgreSQL хранит метаданные и очередь, файлы лежат в общем volume. Миграции применяются отдельным контейнером до запуска API и worker. Redis, Celery и PostGIS не требуются. Одновременный запуск второго worker защищён `advisory lock`.

Компонент	Назначение и источники
Python 3.12, FastAPI, Pydantic	HTTP API и схемы
PostgreSQL 16, SQLAlchemy, asyncpg, Alembic	Модели и миграции
ezdxf	Чтение, нормализация и экспорт CAD
Shapely	Геометрия и зоны ограничений
LangChain	Оptionальное композиционное планирование
React 18, TypeScript, Vite, OpenLayers	Веб-интерфейс

Точные зависимости зафиксированы в `poetry.lock` и `frontend/package-lock.json`.

Помощник: настройка LLM

Для внешнего провайдера задайте в `.env`:

```
AGENT_ENABLED=true
LLM_PROVIDER=openai
LLM_MODEL=<идентификатор-доступной-модели>
LLM_API_KEY=<ключ>
ALLOW_EXTERNAL_LLM=true
```

Для OpenAI-compatible сервера используйте `LLM_PROVIDER=openai_compatible` и `LLM_BASE_URL` с окончанием `/v1`. Модель должна поддерживать используемые агентом инструменты и структурированные ответы. Проверить провайдера нужно до демо.

При прямом локальном запуске `loopback-адрес` (`127.0.0.1 / localhost`) не требует разрешения внешней передачи. В Docker `127.0.0.1` означает сам контейнер. Для сервера на хосте Docker Desktop можно использовать `host.docker.internal`; этот адрес текущая проверка считает внешним и требует

ALLOW_EXTERNAL_LLM=true. На Linux доступный контейнеру адрес сервера настраивается отдельно.

Полный DXF не передаётся модели. При включении внешнего режима передаются необходимые для задачи сводки и кандидаты. Не включайте режим без разрешения на передачу данных участка. После изменения .env пересоздайте оба сервиса:

```
docker compose up -d --force-recreate api worker
```

API

Префикс по умолчанию - /api; актуальные тела запросов и ответы описаны в Swagger.

Метод и путь после /api	Действие
POST /projects/	Создать проект
POST /projects/{id}/files/	Загрузить исходник
POST /projects/{id}/analyze	Поставить анализ в очередь
PUT /projects/{id}/config	Сохранить подтверждённую конфигурацию
POST /projects/{id}/plans	Генерация без LLM
POST /projects/{id}/agent/autoplan	Генерация с помощником
GET /jobs/{id}	Статус фоновой задачи
GET /projects/{id}/plans/current	Текущий план
POST /projects/{id}/plans/{plan_id}/validate	Повторная проверка
GET /projects/{id}/plans/{plan_id}/report	Объяснимый отчёт
POST /projects/{id}/plans/{plan_id}/exports	Экспорт; ?draft=true - явный черновик
GET /projects/{id}/artifacts/{artifact_id}	Скачать артефакт

Долгие операции возвращают HTTP 202 и задачу. Перед правкой, повторной проверкой и экспортом получите план, прочитайте ETag и передайте его в If-Match, например "1". Без ревизии возвращается 428; при конфликте версий - 409.

Локальная разработка и проверки

Для разработки нужны Python 3.12, Poetry 1.8.1 (версия Dockerfile), Node.js 20 и PostgreSQL. .env должен существовать и содержать актуальные параметры БД.

```
poetry install --no-root
docker compose up -d db
poetry run alembic upgrade head
poetry run uvicorn app.main:backend_app --host 127.0.0.1 --port 8000
```

В другом терминале:

```
poetry run python -m app.worker
```

Frontend в отдельном терминале:

```
cd frontend
npm ci
npm run dev
```

Адрес разработки - `localhost:5173/static/`. Vite проксирует `/api` на порт 8000. `npm run build` проверяет TypeScript и собирает frontend в `app/static`, который раздаёт FastAPI.

Проверки из корня проекта:

```
docker compose --profile test up -d test_db
poetry run pytest
poetry run ruff check app tests
poetry run ruff format --check app tests
npm --prefix frontend run build
```

`TEST_DATABASE_URL` должен указывать на отдельную БД с `test` в имени: тестовые fixtures пересоздают её таблицы. При изменении логина/пароля PostgreSQL обновите тестовый URL. `pytest` включает coverage с порогом 80%; успешные функциональные тесты сами по себе не означают прохождение этого порога.

Команды выше - инструкция воспроизведения, а не заявление о выполнении всех проверок на машине эксперта. Фактический статус перед сдачей - в чек-листе.

Ограничения и эксплуатация

- Полное покрытие НПА, охранных зон, климата и почв не реализовано. Подбор по справочнику не доказывает агрономическую пригодность каждой посадки.
- В MVP генерируются деревья и кустарники. Газоны, травянистые покрытия, элементы скверов и фотореалистичные визуализации не реализованы.
- XREF, неподдерживаемые объекты, неверные единицы и неполная классификация требуют решения проектировщика. Большой DXF сам по себе не является готовым входом.
- Preview - диагностическая карта, не полноценный CAD. По умолчанию ограничены 2 000 объектов, 2 000 зон и около 200 000 координат; упрощение отображения не меняет геометрию расчёта. Неполноту preview нужно учитывать при просмотре.
- Лимит загрузки backend по умолчанию - 300 МБ, артефакта - 500 МБ; параметры доступны в `.env.example`. Текущая подсказка UI «200 МБ» не совпадает с лимитом backend.
- Один worker обрабатывает задачи последовательно. Авторизация и промышленный многопользовательский режим не реализованы; приложение рассчитано на локальный запуск.
- Работа на МосТех.ОС и визуальная совместимость результата с целевым CAD должны быть проверены отдельно. Dockerfile не является подтверждением такой проверки.

Диагностика: `docker compose logs -f api worker`. Sentry опционален: без `SENTRY_DSN` выключен; `SENTRY_ENVIRONMENT`, `SENTRY_RELEASE`, `SENTRY_ENABLE_LOGS` и `SENTRY_TRACES_SAMPLE_RATE` заданы в `.env.example`. API и worker различаются тегом `service`; передача тел запросов и DXF отключена конфигурацией приложения.

Логотип и фирменное изображение предоставлены командой проекта.